Cyberscope
A TAC Security Company

Audit Report
NUMMUS Vault

October 2026

Repository	https://github.com/028477-bot/nummus-vault/
Commit	07faa59abc0cef1b4bfa2e3a8297db6584f72624
Audited by	© cyberscope

Table of Contents

Table of Contents 2
Risk Classification 3
Overview 4
Findings Breakdown 5
IVAP - Inconsistent Validation Across Paths 6
Description 6
Recommendation 6
Team Update 6
SCLSC - SOL Claims Lack Settlement Conversion 7
Description 7
Recommendation 7
Team Update 7
USEL - Unbounded Swap Execution Limits 8
Description 8
Recommendation 8
Team Update 8
UTRC - Unsupported Token-2022 Reward Collection 9
Description 9
Recommendation 9
AFSF - Adaptive Fee Settlement Failure 10
Description 10
Recommendation 10
ARNR - Authority Rotation No Recovery 11
Description 11
Recommendation 11
Team Update 11
PCR - Program Centralization Risk 12

Description 12
Recommendation 12
Team Update 12
RRPL - Receipt Rent Permanently Locked **13**
Description 13
Recommendation 13
Team Update 13
RRTPN - Reward Release Transfers Position NFT **14**
Description 14
Recommendation 14
SPDFE - Slippage Policy Disconnected From Execution **15**
Description 15
Recommendation 15
Team Update 15
TRWI - Tick Range Width Ineffective **16**
Description 16
Recommendation 16
Team Update 16
WRSC - Withdrawal Receipt Seed Collision **17**
Description 17
Recommendation 17
Team Update 17
Summary **18**
Disclaimer **19**
About Cyberscope **20**

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
Critical	Highly Likely / High Impact
Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
Minor / Informative	Unlikely / Low to no Impact

Overview

Nummus Vault consists of a single Solana program that accepts SOL deposits, records user balances, and supports authority-managed liquidity operations through Orca Whirlpools.

Vault Deposits

Users deposit native SOL into a program-controlled vault. Each deposit increases the depositor's on-chain principal balance and creates a permanent receipt identified by the user's wallet and deposit ID. User positions track balances and deposit and withdrawal counts.

Withdrawals

The designated vault authority executes withdrawals, sending SOL exclusively to the wallet that owns the position. Withdrawals reduce the user's recorded balance, cannot exceed that balance, and must preserve the vault's required SOL reserve. Each withdrawal creates a permanent receipt with a globally unique request ID to prevent duplicate settlement.

Orca Liquidity Management

The vault authority can open one Orca liquidity position at a time, increase or decrease its liquidity, and close it. Operations use the configured Whirlpool and token accounts, with checks on account ownership, token mints, position identity, and tick arrays. Position ranges must fit within configured tick boundaries, while liquidity changes pass authority-supplied token maximums or minimums to Orca.

Native SOL Handling

Before adding liquidity, the program can wrap vault SOL into its associated wrapped-SOL account. Removing liquidity returns assets to the vault's token accounts. Closing the Orca position also closes the wrapped-SOL account, returning its SOL to the main vault.

Liquidity Fees

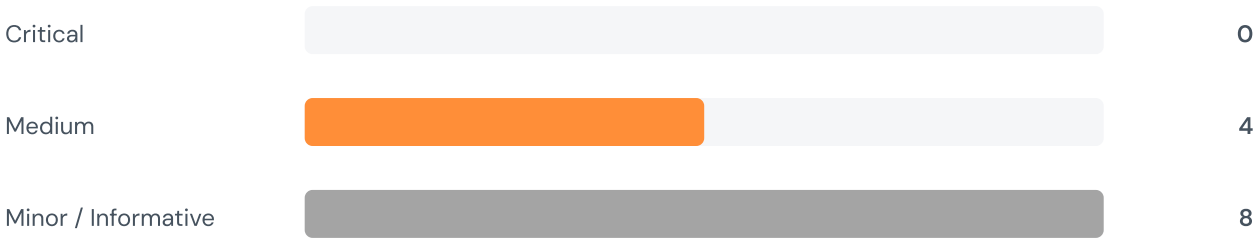
The vault authority can update the Orca position's fee and reward accounting and collect trading fees into vault-controlled token accounts. User balances track deposited principal less withdrawals; the program does not calculate a share price or automatically credit collected fees to individual users.

Administration

The administrator configures tick boundaries and the declared slippage ceiling, and can independently pause deposits and liquidity operations. These pause flags do not block withdrawals. Both the administrator and vault authority can be replaced through a two-step process in which the administrator proposes a successor and the nominated signer accepts the role.

Findings Breakdown

12 Issues Found



Severity	Unresolved	Acknowledged	Resolved	Other
Critical	0	0	0	0
Medium	1	3	0	0
Minor / Informative	2	6	0	0

Critical Medium Minor / Informative

Severity	Code	Description	Status
●	IVAP	Inconsistent Validation Across Paths	Acknowledged
●	SCLSC	SOL Claims Lack Settlement Conversion	Acknowledged
●	USEL	Unbounded Swap Execution Limits	Acknowledged
●	UTRC	Unsupported Token-2022 Reward Collection	Unresolved
●	AFSF	Adaptive Fee Settlement Failure	Unresolved
●	ARNR	Authority Rotation No Recovery	Acknowledged
●	PCR	Program Centralization Risk	Acknowledged

●	RRPL	Receipt Rent Permanently Locked	Acknowledged
●	RRTPN	Reward Release Transfers Position NFT	Unresolved
●	SPDFE	Slippage Policy Disconnected From Execution	Acknowledged
●	TRWI	Tick Range Width Ineffective	Acknowledged
●	WRSC	Withdrawal Receipt Seed Collision	Acknowledged

IVAP - Inconsistent Validation Across Paths

Severity	Location	Status
medium	liquidity.rs#L16,516,587,632	acknowledged

Description

The revised program adds reserve enforcement and revalidates the configured tick range before increasing liquidity, addressing two of the inconsistencies originally reported under IVAP. However, both liquidity directions still validate the caller's declared `slippage_bps` without establishing a relationship between that declaration and the token limits forwarded to Orca. The additional execution-bound checks reject certain empty or unlimited values but do not enforce the configured percentage tolerance.

An authorized caller can therefore declare zero slippage while supplying permissive token maxima or low withdrawal minima. Orca enforces those token limits, but execution may exceed the tolerance represented by the vault's slippage check.

RUST

```rust
// check_liquidity_common
require!(
    slippage_bps <= config.max_slippage_bps,
    VaultError::SlippageTooHigh
);

// increase_liquidity_handler
data.extend_from_slice(&args.token_max_a.to_le_bytes());
data.extend_from_slice(&args.token_max_b.to_le_bytes());

// decrease_liquidity_handler
data.extend_from_slice(&args.token_min_a.to_le_bytes());
data.extend_from_slice(&args.token_min_b.to_le_bytes());
```

Recommendation

Consider defining whether slippage protection is an on-chain policy or an obligation of the authorized execution service. If enforced on-chain, the token limits should be validated against an appropriate reference and the configured tolerance across both liquidity directions.

Team Update

The team has acknowledged that this is not a security issue and states: *Response: Explicit execution-service responsibility; no independent on-chain price guarantee.We clarified the responsibility across both liquidity directions. The authorized service selects quotes, freshness requirements and acceptable token maxima/minima. The contract checks its structural and authorization constraints, and Orca enforces the supplied execution limits.*

The retained slippage_bps declaration and configured ceiling are not mathematically linked to an independent reference quote. They must not be interpreted as proof that execution is within that percentage. No oracle-based enforcement has been added. A compromised authorized service can choose harmful limits even when the declared basis-point value passes validation.

SCLSC - SOL Claims Lack Settlement Conversion

Severity	Location	Status
medium	deposit.rs#L94 withdraw.rs#L60,71,91 settlement.rs#L196,241	acknowledged

Description

The program executes authority-approved withdrawals without independently calculating user entitlements or reconciling aggregate outstanding obligations with available backing. Deposits and withdrawals clear the deprecated balance field rather than maintaining nominal user claims. Counter-token conversion and wSOL unwrapping improve liquidity availability, but do not determine withdrawal entitlements. Accounting, profit/loss allocation, and final payout amounts are assigned to the server.

If the server authorizes payouts without reconciling outstanding obligations against total backing across all funding sources, earlier withdrawals can consume funds needed for later payouts. For example, if externally recorded obligations remain 20 SOL after total backing falls to 15 SOL, paying one user 10 SOL leaves only 5 SOL for the remaining obligations. Subsequent withdrawals fail when available native SOL cannot cover the approved amount while preserving the required reserve. This scenario depends on the server's accounting behavior; the contract source alone does not establish that losses remain unallocated or nominal payouts are guaranteed.

RUST

```rust
// deposit_handler: deprecated balance bookkeeping
position.balance_lamports = 0;

// withdraw_handler: available SOL must cover the approved payout
require!(
    crate::logic::withdrawal_keeps_reserve(vault_lamports, rent_exempt_min, amount)?,
    VaultError::RentReserveBreach
);

// Deprecated balance field is cleared after payment
position.balance_lamports = 0;
```

Recommendation

Consider verifying that server-calculated withdrawal entitlements reflect an explicit loss-allocation policy and total backing across all funding sources. Settlement controls should account for outstanding obligations and avoid disproportionately allocating shortfalls to later withdrawals.

Team Update

The team has acknowledged that this is not a security issue and states: *Response: Duplicate contract accounting removed; server accounting and contract execution responsibilities clarified.*

The contract acts like a wallet or vault: it receives, holds and transfers funds. The server handles the accounting, trading decisions, profit/loss calculations and final payout amounts. The contract executes authorized instructions within its security checks; it does not make those financial decisions itself.

The finding describes an earlier implementation that recorded nominal user balances and capped withdrawals against those balances. That duplicated accounting did not match the intended architecture: the server handles user accounting, trading results and the final amount payable, while the contract provides custody and authorized execution.

The vault will also receive funds from other trading activities. Consequently, the configured Orca position is not the sole source of vault funds, and its performance alone does not determine the server-calculated payout. Determining that payout belongs to the server's overall accounting, not to a second accounting model inside this contract.

We removed the deposit-balance payout ceiling and retired the remaining-balance bookkeeping. The server determines the final withdrawal amount and submits it for the required authorization. The contract transfers that exact amount to the position's bound wallet. It does not independently recalculate, reduce or cap the payment at the original deposit. If the available SOL cannot support the full payment while preserving the required reserve, the transaction fails atomically rather than making a partial payment.

The deprecated balance fields retain their names and storage/event layouts for compatibility. Successful deposits and withdrawals write zero to the position field and emit zero in the corresponding event field. Existing untouched accounts may retain historical values until their next successful operation under this version; those values are not withdrawal entitlements. Wallet binding, permanent receipts and cumulative deposit/withdrawal records remain.

This change removes a conflicting contract-side representation of user entitlements; it does not remove the server's accounting responsibility. Profit/loss treatment and determination of the current full-close amount remain server responsibilities. An authorized amount is not a guarantee of the original deposit value.

Please reassess the finding against this division of responsibilities and the vault's multiple funding sources. The absence of contract-side entitlement calculations should not be described as the absence of an assigned accounting responsibility: that responsibility belongs to the server. This explanation describes the operating model; the contract source package alone does not independently verify the server's accounting implementation or establish audit clearance.

USEL - Unbounded Swap Execution Limits

Severity	Location	Status
medium	instructions/settlement.rs#L149,179,196,206,229,230,243,244	acknowledged

Description

The settlement instruction performs an Orca swap of the counter token into wrapped SOL, with execution limits supplied by the vault authority. `validate_swap_bounds` requires the input amount to be nonzero and not the maximum sentinel, requires `minimum_amount_out` to be nonzero, and requires `sqrt_price_limit` to be nonzero and positioned on the correct side of the pool's current price. These checks do not independently bound acceptable economic loss. An authorised caller may therefore declare a minimum output of a single unit and a price limit at the extreme of Orca's permitted range while satisfying the program's input validation.

The handler confirms that exactly the requested input was consumed and independently measures the output delta against the declared minimum. However, the minimum output and price limit are not validated against an independent price reference, leaving acceptable execution quality under the submitting authority's control. Neither pause flag is checked on the settlement path, so pausing deposits or liquidity allocation does not prevent the vault authority from moving funds through otherwise valid settlement swaps, including swaps with permissive execution limits.

RUST

```rust
// instructions/settlement.rs
fn validate_swap_bounds(
    args: &SettleCounterToNativeArgs,
    current: u128,
    a_to_b: bool,
) -> Result<()> {
    require!(
        args.amount_in > 0
            && args.amount_in != u64::MAX
            && args.minimum_amount_out > 0,
        VaultError::InvalidExecutionBounds
    );
    require!(
        args.sqrt_price_limit > 0
            && if a_to_b {
                args.sqrt_price_limit < current
            } else {
                args.sqrt_price_limit > current
            },
        VaultError::InvalidExecutionBounds
    );
    Ok(())
}
```

Recommendation

The team is advised to determine whether execution quality on the settlement path is intended to be an on-chain guarantee or an obligation of the authorised service, and to make the program consistent with that determination. Should it be intended as an on-chain guarantee, the resulting behaviour should hold the property that the worst acceptable execution is bounded by something other than the submitting party's own declaration. Should it be intended as a service obligation, the team is advised to state that explicitly, including in any material presented to depositors, so that the bound is understood to rest with the operator.

Team Update

The team has acknowledged that this is not a security issue and states: *Response: Explicit execution-service responsibility; residual pricing risk disclosed.*

We documented that counter-token conversion relies on the authority's minimum output and explicit swap price limit. The existing checks reject invalid inputs and enforce the submitted limits, including exact input consumption and the measured minimum output. They do not independently establish that those limits represent a fair market price.

We have not added a trusted price reference or an independent bound on acceptable economic loss. Permissive but structurally valid limits remain an operator trust risk. Please assess this as an explicitly disclosed service obligation rather than an on-chain percentage-slippage guarantee.

UTRC - Unsupported Token-2022 Reward Collection

Severity	Location	Status
medium	settlement.rs#L12,33,48,54 orca_cpi.rs#L31,42	unresolved

Description

`CollectReward` pins the token program to legacy SPL Token, while `collect_reward_handler` requires the reward mint to be owned by that program and have a data length of exactly 82 bytes. The Orca instruction allowlist permits `collect_reward` but excludes `collect_reward_v2`, which supports Token-2022 rewards. Consequently, rewards issued under Token-2022 cannot be collected through the vault.

If the configured Whirlpool accrues Token-2022 rewards, those rewards remain uncollectible through the existing instructions. Outstanding reward amounts can also prevent Orca position closure, potentially blocking replacement of the vault's active position.

RUST

```rust
// CollectReward
#[account(
    address = SPL_TOKEN_PROGRAM_ID
        @ VaultError::OrcaAccountConstraintViolation
)]
pub token_program: UncheckedAccount<'info>,

// collect_reward_handler
require_keys_eq!(
    *a.reward_mint.owner,
    SPL_TOKEN_PROGRAM_ID,
    VaultError::InvalidMint
);
require!(a.reward_mint.data_len() == 82, VaultError::InvalidMint);

let mut data = DISC_COLLECT_REWARD.to_vec();
```

Recommendation

Consider aligning reward collection with the token standards supported by the configured pool, including appropriate mint validation, account derivation, and Orca instruction selection. If support remains limited to legacy SPL Token, explicitly document and verify reward compatibility

before allocating liquidity, including the consequences of unsupported rewards for collection and position closure.

AFSF - Adaptive Fee Settlement Failure

Severity	Location	Status
minor	settlement.rs#L143,169,196,239 orca_cpi.rs#L246,263	unresolved

Description

`SettleCounterToNative` declares the Orca oracle account without a mutable constraint, and `settle_counter_to_native_handler` explicitly passes it as read-only in the swap CPI. Orca's legacy swap implementation updates oracle state for adaptive-fee pools and requires this account to be writable. Consequently, otherwise valid settlement swaps against these pools fail with `AccountNotMutable`.

If liquidity removal completes in a separate transaction, the resulting counter-token balance cannot be converted into native SOL through this instruction. Since withdrawals pay native SOL, this can block payouts requiring those funds when available SOL is insufficient. Liquidity removal and settlement submitted in the same transaction roll back together. The impact depends on whether the configured pool uses adaptive fees.

RUST

```rust
// SettleCounterToNative account declaration
pub oracle: UncheckedAccount<'info>,

// Swap CPI account list
meta(&a.oracle, false, false),
```

Recommendation

Consider aligning oracle account permissions with Orca's adaptive-fee requirements throughout the account definition, CPI account metadata, and client instruction construction.

ARNR - Authority Rotation No Recovery

Severity	Location	Status
minor	instructions/admin.rs#L18,22,26,153,154,166	acknowledged

Description

Thee code introduced `validate_rotation_consent`, which requires that replacement of the vault authority be signed by the current administrator and by the outgoing vault authority. Both are supplied as optional signers and both must be present and must match stored configuration. The condition is correct for its purpose, and the two-step escalation path identified previously is closed by it.

It also removes the only means by which the role could be replaced without the participation of its current holder. Because every withdrawal requires the vault authority's signature, a vault authority key that becomes permanently unavailable leaves every recorded user balance unsettleable with no remedy available on chain.

RUST

```rust
// instructions/admin.rs — replacement requires the holder being replaced
fn validate_rotation_consent(
    admin: Pubkey, vault_authority: Pubkey,
    consenting_admin: Option<Pubkey>, outgoing_authority: Option<Pubkey>,
) -> Result<()> {
    require!(consenting_admin == Some(admin), VaultError::UnauthorizedAdmin);
    require!(outgoing_authority == Some(vault_authority), VaultError::UnauthorizedVaultAuth
    Ok(())
}
```

Recommendation

The team is advised to record the position explicitly rather than leave it to be inferred from the rotation conditions: that the availability of every recorded user balance depends on the continued availability of a single key, and that the program provides no succession for it. The custody arrangements for that key, and any procedure for its replacement, now sit entirely outside the program and warrant documentation proportionate to that dependency.

Team Update

The team has acknowledged that this is not a security issue and states: *Response: Recovery limitation explicitly documented; no lost-key bypass added.*

Normal authority replacement continues to require outgoing-authority consent. The program provides no independent succession mechanism if that authority becomes permanently unavailable.

Signing availability and recovery depend on the external custody arrangements. The intended exceptional recovery route, if those arrangements fail, is a separately reviewed program upgrade under the distinct 4-of-6 root-controlled upgrade authority. This is not an existing on-chain recovery instruction, and the published package does not prove that the required custody and upgrade controls are configured or operational.

PCR - Program Centralization Risk

Severity	Location	Status
minor	admin.rs#L33,62,105,157 withdraw.rs#L45,60,71 liquidity.rs#L195,365,589 settlement.rs#L196	acknowledged

Description

The vault concentrates operational control in the `admin` and `vault_authority` roles. The administrator controls deposit and liquidity pause flags, configurable tick bounds, maximum slippage, and authority rotation proposals. The vault authority must authorize every withdrawal and liquidity operation, leaving depositors without an independent withdrawal path. Loss, unavailability, or refusal to act by this authority can prevent withdrawals and strategy management. Normal vault-authority rotation requires signatures from the current administrator, outgoing vault authority, and proposed replacement; replacement acceptance alone cannot recover access when the outgoing authority is unavailable.

Withdrawal amounts are determined outside the program and are **not capped by recorded deposits or user balances**. Although destinations remain bound to the associated position owner and payouts must preserve the required reserve, the operator can theoretically drain spendable vault liquidity by authorizing excessive withdrawals to an operator-controlled wallet with a valid user position, including SOL recovered through liquidity removal and successful settlement. Compromise of privileged keys can also enable disruptive configuration changes or economically harmful liquidity decisions. These capabilities represent an explicit custody and trust dependency.

RUST

```rust
// AdminOnly
#[account(
    mut,
    seeds = [CONFIG_SEED],
    bump = config.config_bump,
    constraint = config.version == CONFIG_VERSION
        @ VaultError::ConfigVersionMismatch,
    has_one = admin @ VaultError::UnauthorizedAdmin,
)]
pub config: Account<'info, Config>,
pub admin: Signer<'info>,

// Withdraw
#[account(
    constraint = vault_authority.key() == config.vault_authority
        @ VaultError::UnauthorizedVaultAuthority
)]
pub vault_authority: Signer<'info>,

// withdraw_handler
// The authority approves the exact payout; this custody program does not
// calculate entitlements or cap approved payouts at deposited principal.
require!(
    crate::logic::withdrawal_keeps_reserve(vault_lamports, rent_exempt_min, amount)?,
    VaultError::RentReserveBreach
);
```

Recommendation

Consider distributing privileged control across a multisignature or governance mechanism and requiring additional approvals for high-impact operations. Withdrawal authorization may benefit from independently verifiable entitlements, payout limits, and monitoring. Consider documenting and validating authority recovery procedures, particularly where normal rotation depends on an unavailable outgoing authority, and clearly communicating the operator's custody powers to depositors.

Team Update

The team has acknowledged that this is not a security issue and states: *Response: Intentional custody dependency disclosed; no claim of trustless redemption.*

We explicitly describe the administrator and vault-authority powers and the absence of an independent depositor withdrawal path. Availability, correct authorization and responsible trading decisions continue to depend on the operator and its signing infrastructure.

The intended Turnkey approval model separates constrained liquidity automation, withdrawals requiring automation plus manual approval, and privileged/root operations requiring 4-of-6 approval. These are external signing controls, not a quorum enforced by the vault program itself. Included policy templates are not evidence of live activation or successful enforcement. Centralization and upgrade-authority risks remain subject to your assessment.

RRPL - Receipt Rent Permanently Locked

Severity	Location	Status
minor	deposit.rs#L36,38 withdraw.rs#L35,37	acknowledged

Description

Both receipt account types are created with `init` and are never closed, and the program exposes no instruction capable of closing one. The rent paid to render each receipt rent-exempt is therefore locked permanently. At current rent parameters a deposit receipt locks approximately 0.00135 SOL and a withdrawal receipt approximately 0.00157 SOL, paid in each case by the user or the payer rather than by the vault.

The accounts serve solely as replay markers and hold no state that must persist beyond the point at which replay of the identifier ceases to be a concern. For a vault expecting frequent small deposits the accumulated cost represents a material and largely invisible reduction in user returns, scaling with transaction count rather than with assets under management. The design is safe and the cost is bounded per operation; it is larger than the function requires.

RUST

```
#[account(
    init,
    payer = depositor,
    space = DepositReceipt::LEN,
    seeds = [DEPOSIT_RECEIPT_SEED, depositor.key().as_ref(), &deposit_id.to_le_bytes()],
    bump,
)]
pub deposit_receipt: Account<'info, DepositReceipt>,
```

Recommendation

Consider adding a permissioned instruction allowing a receipt to be closed once its identifier can no longer be replayed, returning the rent to its original payer. Where the off-chain issuer allocates monotonically increasing identifiers, a single high-water mark recorded in the user's position account would supersede individual receipts and eliminate the per-operation cost.

Should receipts be retained for their audit value, consider documenting the cost and bearing it from vault capital rather than from the depositor, so that the expense of internal bookkeeping is not deducted from user returns without disclosure.

Team Update

The team has acknowledged that this is not a security issue and states: *Response: Permanent receipts deliberately retained; cost disclosed.*

We retain receipts as permanent replay evidence. Closing a receipt without a replacement replay mechanism would permit identifier reuse. The current hashed and potentially out-of-order identifiers do not support a simple monotonic high-water-mark substitute.

The depositor funds the deposit receipt, and the withdrawal payer funds the withdrawal receipt. At the report's rent parameters, approximately 0.00135 SOL and 0.00157 SOL respectively remain locked per receipt; actual amounts depend on cluster rent parameters. Withdrawal receipt rent is not deducted from the exact authorized payout.

We have not implemented receipt closure, rent refunds or a change that makes the vault bear these costs.

RRTPN - Reward Release Transfers Position NFT

Severity	Location	Status
minor	reward_release.rs#L39,47,77 liquidity.rs#L228,269,341	unresolved

Description

`release_reward_handler` excludes the configured trading-pair mints and native SOL from reward releases but does not exclude `config.position_mint`. If the configured Whirlpool's reward authority initializes an unused reward slot with the active position NFT mint, the canonical reward source resolves to the vault's position-token account. An administrator-authorized release of one token can consequently transfer the position NFT to the administrator's canonical token account.

The NFT authorizes management of the underlying Orca position. Transferring it permits the recipient to manage liquidity directly through Orca, outside the vault's execution restrictions, while the vault configuration continues referencing the emptied position-token account. This is a conditional issue requiring both privileged reward configuration and an administrator's release signature; applicability to the deployed pool remains unverified.

RUST

```
let (mint, _) = read_reward(&a.whirlpool.to_account_info(), reward_index)?;
require!(
    mint != c.token_mint_a && mint != c.token_mint_b && mint != NATIVE_SOL_MINT,
    VaultError::InvalidMint
);
require_keys_eq!(
    a.source.key(),
    canonical_ata(a.vault_sol.key, &mint),
    VaultError::InvalidAssociatedTokenAccount
);
```

Recommendation

Consider explicitly excluding the active position NFT mint and its custody account from reward releases. Reward eligibility should distinguish distributable assets from tokens granting control over vault-held positions, independently of external reward metadata.

SPDFE - Slippage Policy Disconnected From Execution

Severity	Location	Status
minor	liquidity.rs#L16,313,486,525,548,561,564,565,587,589,590	acknowledged

Description

`increase_liquidity_handler` validates the caller-supplied `slippage_bps` against `config.max_slippage_bps`, but does not establish any relationship between this value and `token_max_a` or `token_max_b`. Orca receives the requested liquidity amount and token maxima, which determine the actual execution limits; the declared basis-point value is neither forwarded nor used to derive these limits.

An authorized caller can therefore declare zero slippage while supplying permissive, fundable token maxima. Once liquidity execution is reachable, adverse price movement may cause the operation to consume amounts exceeding the intended tolerance while still satisfying the configured slippage check. This does not bypass Orca's token maxima or establish unauthorized access; it makes enforcement of the vault's slippage policy dependent on the caller correctly constructing those maxima.

RUST

```rust
// check_liquidity_common
require!(
    slippage_bps <= config.max_slippage_bps,
    VaultError::SlippageTooHigh
);

// increase_liquidity_handler
data.extend_from_slice(&DISC_INCREASE_LIQUIDITY);
data.extend_from_slice(&args.liquidity_amount.to_le_bytes());
data.extend_from_slice(&args.token_max_a.to_le_bytes());
data.extend_from_slice(&args.token_max_b.to_le_bytes());

check_liquidity_common(&ctx.accounts.config, args.slippage_bps)?;
// ...
data.extend_from_slice(&args.token_min_a.to_le_bytes());
data.extend_from_slice(&args.token_min_b.to_le_bytes());
```

Recommendation

Consider deriving or validating the token maxima against a trusted reference quote for the requested liquidity and the configured slippage tolerance. The reference should account for freshness, price manipulation, and rounding so that the basis-point policy constrains the actual execution limits.

Any remediation should cover both liquidity directions, and the report should state that requirement explicitly so that a partial fix is not accepted as complete. Consider establishing that the declared tolerance bounds the worst execution the vault can accept on either path, and that the bound is not determinable by the party submitting the operation. Where a binding relationship is judged impractical, consider whether retaining an argument and a check that constrain nothing is preferable to removing them, given that their presence represents to depositors and reviewers a guarantee the program does not provide.

Team Update

The team has acknowledged that this is not a security issue and states: *Response: Misleading guarantee clarified; independent enforcement not added.*

As with IVAP, we explicitly distinguish the retained slippage_bps declaration from the token limits that actually govern execution. The declaration and ceiling remain for compatibility/authorization, not as proof of an independently enforced percentage tolerance.

This clarification applies to both liquidity increases and decreases. No trusted quote, freshness oracle or on-chain derivation tying those token limits to the declared percentage has been added. Correct construction of the limits remains the authorized service's responsibility.

TRWI - Tick Range Width Ineffective

Severity	Location	Status
minor	constants.rs#L30 logic.rs#L25,32 initialize.rs#L71 admin.rs#L74	acknowledged

Description

The revised program corrects `MAX_TICK_RANGE_WIDTH` to `887_272` and validates that configured endpoints remain within Orca's legal bounds of `-443_636` to `443_636`. This resolves acceptance of impossible configured ranges. However, the width limit equals the entire legal tick span, so every ordered range satisfying the endpoint checks also satisfies the width check.

RUST

```
pub const MAX_TICK_RANGE_WIDTH: i32 = 887_272;

require!(
    min_tick >= -443_636 && max_tick <= 443_636,
    VaultError::TickRangeOutOfBounds
);
require!(min_tick < max_tick, VaultError::TickRangeOutOfBounds);
require!(
    (max_tick as i64 - min_tick as i64) <= MAX_TICK_RANGE_WIDTH as i64,
    VaultError::TickRangeOutOfBounds
);
```

Recommendation

Consider documenting the intended maximum strategy width. If a narrower allocation limit is required, configure and enforce that limit independently of Orca's absolute endpoint bounds.

Team Update

The team has acknowledged that this is not a security issue and states: *Response: Full protocol-domain support explicitly documented.*

Support for the full legal Orca tick span is intentional. The configured endpoints remain enforced on-chain, and the execution service chooses its strategy range within them.

The absolute maximum-width constant is a protocol-domain check, not an additional narrow-strategy limit. We have not introduced a separate, narrower maximum-width policy and do not claim that the existing constant provides one.

WRSC - Withdrawal Receipt Seed Collision

Severity	Location	Status
minor	deposit.rs#L39 withdraw.rs#L38,108	acknowledged

Description

Replay protection for both deposits and withdrawals relies on creating a receipt account whose address derives from the request identifier, so that a repeated request collides with an existing account and fails. The deposit receipt seeds include the depositor's public key and are therefore scoped per user. The withdrawal receipt seeds comprise only the constant and the `request_id`, placing every user's withdrawal identifiers in a single global namespace, although the receipt body records the `owner` field.

Where the off-chain issuer allocates withdrawal identifiers per user, which the deposit derivation would naturally suggest, the first user to settle identifier *N* renders that identifier permanently unusable for every other user, and the collision can be resolved only by renumbering. A secondary consequence is that the address of a given user's withdrawal cannot be derived from the owner and identifier by a client or indexer, unlike the deposit receipt. The behaviour is safe where the issuer guarantees globally unique identifiers; the program itself does not establish that property, and the asymmetry with the deposit path suggests it was not intended.

RUST

```
seeds = [DEPOSIT_RECEIPT_SEED, depositor.key().as_ref(), &deposit_id.to_le_bytes()],

seeds = [WITHDRAWAL_RECEIPT_SEED, &request_id.to_le_bytes()],
...
receipt.owner = position.owner;
```

Recommendation

Consider adding the position owner to the withdrawal receipt seeds, mirroring the deposit receipt derivation, so that replay protection is scoped per user and withdrawal addresses become derivable from the owner and identifier alone. The change affects a PDA derivation and therefore requires migration if applied to a program with live receipts.

Should a single global sequence of request identifiers be the intended design, consider documenting the off-chain issuer as the sole authority for allocating them, together with its obligation to guarantee global uniqueness, so that the dependency between on-chain safety and off-chain discipline is explicit.

Team Update

The team has acknowledged that this is not a security issue and states: *Response: Global namespace retained, with issuer responsibility and collision handling made explicit.*

Withdrawal receipt seeds remain globally scoped and unchanged, preserving existing receipt addresses and replay protection. The service is the sole identifier issuer; identifiers must be unique across users of the program, not allocated as per-user counters.

The integration uses identifiers incorporating position, wallet and signed authorization nonce, together with a global unique database index. Hashing alone is not a mathematical guarantee of uniqueness. Collision/retry handling compares the intended owner, position, destination, program and exact amount. A different intent produces an explicit conflict rather than being treated as an already-processing withdrawal. An occupied identifier for another intent must be replaced before signing.

Settlement also requires the receipt owner, destination and amount to match the intended withdrawal. These issuer and database controls are application-side measures, not a new on-chain guarantee of global identifier issuance. We have not changed the PDA seed scheme.

Summary

Nummus Vault implements a Solana-based SOL vault featuring on-chain user balance tracking, authority-managed withdrawals, permanent transaction receipts, and integration with Orca Whirlpools for liquidity management and fee collection. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an “endorsement” or “disapproval” of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any “product” or “asset” created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



The Cyberscope team

cyberscope.io